

```

"""
DecodeLabs Project 1: Data Cleaning &
Preparation
Author: [Your Name]
Description: Cleans the raw orders dataset
– handles missing values,
            removes duplicates, fixes
formatting – and auto-generates
            a Change Log of every action
taken.
"""

import pandas as pd
import numpy as np
from datetime import datetime

#
=====
=====
# STEP 0: SETUP – Change log tracker
#
=====
=====
change_log = []

def log_change(change_id, description,
impact, status="Resolved"):
    change_log.append({
        "Change ID": change_id,
        "Description": description,

```

```

        "Impact": impact,
        "Status": status
    })

#
=====
=====
# STEP 1: LOAD THE RAW DATA
#
=====
=====
INPUT_FILE = "raw_dataset.csv"    # <--
change this to your actual filename
OUTPUT_FILE = "cleaned_dataset.csv"

df = pd.read_csv(INPUT_FILE)
original_rows = len(df)

print("=== INITIAL AUDIT ===")
print(f"Shape: {df.shape}")
print(f"\nMissing values per
column:\n{df.isnull().sum()}")
print(f"\nData types:\n{df.dtypes}")
print(f"\nDuplicate OrderIDs:
{df.duplicated(subset='OrderID').sum()}")

#
=====
=====
# STEP 2: REMOVE DUPLICATES
#
=====
=====
before = len(df)

```

```

df = df.drop_duplicates(subset="OrderID",
keep="first")
removed = before - len(df)

log_change(
    "CR001",
    "Removed duplicate rows based on
OrderID",
    f"Removed {removed} duplicate
record(s)",
)

#
=====
=====
# STEP 3: HANDLE MISSING VALUES
#
=====
=====

# --- Numeric columns: fill with median ---
numeric_cols = ["Quantity", "UnitPrice",
"ItemsInCart", "TotalPrice"]
for col in numeric_cols:
    if col in df.columns:
        missing = df[col].isnull().sum()
        if missing > 0:
            median_val = df[col].median()
            df[col] =
df[col].fillna(median_val)
            log_change(
                f"CR_{col}_NA",
                f"Imputed missing '{col}'
values using median ({median_val})",

```

```

        f"Filled {missing} missing
value(s)",
        )

# --- Text columns: fill with mode (most
frequent value) ---
text_cols = ["Product", "ShippingAddress",
"PaymentMethod", "OrderStatus",
            "CouponCode",
"ReferralSource"]
for col in text_cols:
    if col in df.columns:
        missing = df[col].isnull().sum()
        if missing > 0:
            mode_val = df[col].mode()[0]
            df[col] =
df[col].fillna(mode_val)
            log_change(
                f"CR_{col}_NA",
                f"Imputed missing '{col}'
values using mode ('{mode_val}')",
                f"Filled {missing} missing
value(s)",
                )

#
=====
=====
# STEP 4: FIX DATE FORMAT -> ISO 8601
(YYYY-MM-DD)
#
=====
=====
if "Date" in df.columns:

```

```

        before_invalid = df["Date"].copy()
        df["Date"] = pd.to_datetime(df["Date"],
errors="coerce", dayfirst=False)
        invalid_dates =
df["Date"].isnull().sum()
        df["Date"] =
df["Date"].dt.strftime("%Y-%m-%d")

        log_change(
            "CR_DATE",
            "Standardized all dates to ISO 8601
format (YYYY-MM-DD)",
            f"Converted {len(df)} date entries;
{invalid_dates} unparseable date(s)
flagged",
        )

#
=====
=====
# STEP 5: CLEAN TEXT FORMATTING (trim
whitespace, fix casing)
#
=====
=====
for col in text_cols:
    if col in df.columns:
        df[col] =
df[col].astype(str).str.strip().str.title()

log_change(
    "CR_TEXT",
    "Trimmed whitespace and standardized
casing for text columns",

```

```

        f"Applied to columns: {'',
        '.join(text_cols)}",
    )

#
=====
=====
# STEP 6: FIX NUMERIC PRECISION (2 decimal
places)
#
=====
=====
price_cols = ["UnitPrice", "TotalPrice"]
for col in price_cols:
    if col in df.columns:
        df[col] = df[col].round(2)

log_change(
    "CR_NUMERIC",
    "Rounded price columns to 2 decimal
places",
    f"Applied to columns: {'',
    '.join(price_cols)}",
)

#
=====
=====
# STEP 7: RECALCULATE TOTALPRICE (optional
sanity check)
#
=====
=====
if all(c in df.columns for c in

```

```

["Quantity", "UnitPrice", "TotalPrice"]):
    expected_total = (df["Quantity"] *
df["UnitPrice"]).round(2)
    mismatches = (df["TotalPrice"] !=
expected_total).sum()
    if mismatches > 0:
        log_change(
            "CR_TOTALCHECK",
            "Flagged rows where TotalPrice
did not match Quantity x UnitPrice",
            f"{mismatches} mismatched
row(s) found (not auto-corrected – review
manually)",
            status="Needs Review"
        )

#
=====
=====
# STEP 8: VERIFICATION GATE (must be zero
before submission)
#
=====
=====
dup_check =
df.duplicated(subset="OrderID").sum()
date_check = df["Date"].apply(
    lambda x: not
pd.notnull(pd.to_datetime(x, format="%Y-%m-
%d", errors="coerce"))
).sum() if "Date" in df.columns else 0

print("\n=== VERIFICATION GATE ===")
print(f"Duplicate OrderIDs remaining:

```

```

{dup_check}")
print(f"Incorrectly formatted dates
remaining: {date_check}")

log_change(
    "CR_VERIFY",
    "Final verification check for
duplicates and date formatting",
    f"Duplicates: {dup_check} | Bad dates:
{date_check}",
    status="Passed" if dup_check == 0 and
date_check == 0 else "Failed"
)

#
=====
=====
# STEP 9: EXPORT CLEANED DATA
#
=====
=====
df.to_csv(OUTPUT_FILE, index=False)
print(f"\nCleaned dataset saved to:
{OUTPUT_FILE}")
print(f"Original rows: {original_rows} ->
Final rows: {len(df)}")

#
=====
=====
# STEP 10: EXPORT CHANGE LOG AS CSV (for
PDF conversion later)
#
=====

```



```
=====
log_df = pd.DataFrame(change_log)
log_df.to_csv("change_log.csv",
index=False)
print("\nChange log saved to:
change_log.csv")
print(log_df.to_string(index=False))
```